



CLAUDE CODE · FUNDAMENTOS · AULA 6

Prompts Eficazes

para código

A diferença entre um prompt mediano e um que gera código perfeito na primeira tentativa. A qualidade do output começa na qualidade do input.

contexto

few-shot

restrições

iteração



Prompts *importantam*

O Claude é tão bom quanto o que você pede.

✗ Prompt vago

```
"Faça uma função de login"
```

- ✗ Linguagem inventada (Python? JS?)
- ✗ Sem validação de entrada
- ✗ Hash de senha inseguro
- ✗ Sem tratamento de erros
- ✗ Retorno desconhecido

✓ Prompt preciso

```
"login(email, password) em Python 3.11:  
valide email com regex, use bcrypt,  
retorne Dict com {token, user_id}  
ou lance AuthError"
```

- ✓ Assinatura exata
- ✓ Validação e bcrypt
- ✓ Type hints + exceção custom
- ✓ Docstring com exemplos

Anatomia do prompt *perfeito*

5 componentes — nem sempre todos, mas saiba quando usar cada um.

01 Objetivo

obrigatório

Verbo de ação: crie, refatore, analise.

02 Contexto

obrigatório

Stack, versões, frameworks, padrões.

03 Restrições

opcional

O que NÃO fazer / limites.

04 Formato

opcional

Tipo de retorno, docstring, testes.

05 Exemplos

opcional

Few-shot: "siga o padrão de `get_user()`".

O Claude aprende muito mais com **exemplos** do que com descrições

Contexto é *tudo*

O Claude não sabe nada do seu projeto além do que você conta.

1 Sem contexto

— resultado inconsistente "Adicione validação de email"

2 Contexto básico

— correto, mas pode precisar de ajustes "Em Python com Pydantic v2, adicione validação de email no modelo User. Use EmailStr."

3 Contexto completo

— código exato, sem ajustes "Em src/models/user.py, adicione EmailStr ao campo email de UserCreate (Pydantic v2). Siga o padrão de PhoneStr. Atualize UserUpdate. Não altere UserRead."

Ensinar com *exemplos*

✗ Zero-shot

```
"Crie um endpoint de criação  
de usuário."
```

- ✗ Estrutura de resposta inventada
- ✗ Validações por suposição
- ✗ Status codes que divergem
- ✗ Padrão de erro diferente

✓ Few-shot

```
"Crie POST /users seguindo o  
padrão de POST /products:  
- 201 com UserResponse  
- Erros: 422, 409  
- Use create_record() de db.py"
```

- ✓ Mesma estrutura de resposta
- ✓ Mesmos status codes
- ✓ Mesmo padrão de erros
- ✓ Reutiliza funções existentes

Diga o que *NÃO* fazer

Restrições claras evitam que o Claude "adivinhe" e mude o que você não quer.



Escopo de arquivo

✗ "Refatore a autenticação" → ✓ "Refatore **apenas** `auth/validators.py`. Não altere `models` nem `routes`."



Interface pública

✗ "Melhore a performance" → ✓ "Otimize internamente. A assinatura deve permanecer **idêntica**."



Dependências

✗ "Adicione validação" → ✓ "Use **apenas Zod** (já no projeto). Não adicione novas dependências."



Testes

✗ "Corrija os bugs" → ✓ "Corrija **sem quebrar** os testes existentes. Não apague testes."

Técnicas de *prompting*

Expanda cada técnica.

Chain of Thought — peça o raciocínio antes

Peça ao Claude para raciocinar antes de escrever. Melhora tarefas complexas: *"Antes de implementar, liste os edge cases que esta função precisa tratar. Depois implemente."*

Decomposição — quebre tarefas grandes

Em vez de um prompt gigante, quebre em prompts menores e sequenciais: *"1. Mostre só a assinatura das funções. 2. Após aprovação, implemente."*

Iteração guiada — refine progressivamente

Comece simples e use as respostas para enriquecer o próximo prompt: *"O código está bom. Agora adicione tratamento de erros para X, Y e Z."*

Role prompting — atribua um papel

Especifique o nível de expertise para alinhar estilo e profundidade: *"Como um engenheiro sênior focado em performance, revise este código e sugira otimizações."*

Prompts *por tarefa*

Clique nas abas para ver modelos prontos.



Feature



Bug



Refatorar



Review

```
Implemente [funcionalidade] em [arquivo].
Stack: [linguagem+versão, frameworks].
Siga o padrão de [exemplo existente].
Retorne [tipo/formato]. Inclua [docstring/testes].
Não altere [o que preservar].
```

```
Corrija o bug em [função/arquivo].
Comportamento atual: [o que acontece].
Comportamento esperado: [o que deveria].
Não altere a assinatura pública.
Mantenha os testes existentes passando.
```

```
Refatore [função/arquivo/módulo].
Objetivo: [clareza/performance/testabilidade].
Manter interface pública idêntica.
Seguir convenções: [padrão do projeto].
Após refatoração, rode os testes.
```

Teste *rápido*

Qual prompt tem mais chance de gerar o código certo na primeira tentativa?

A "Melhore a função `get_users()`"

B "Otimize `get_users()` em `src/db.py` internamente. Mantenha a assinatura idêntica. Reduza as N+1 queries usando join. Testes devem continuar passando."

C "Deixe `get_users()` mais rápida"

D "Arrume os problemas de performance"

Resposta: B. Tem **objetivo** (otimizar), **alvo** (`src/db.py`), **restrição** (assinatura idêntica), **direção técnica** (N+1 → join) e **critério de sucesso** (testes passando). As outras são verbos vagos — o Claude teria que adivinhar o que fazer, o arquivo e o critério de "melhor".



RESUMO DA AULA 6

Prompt preciso = *código perfeito*

A diferença entre 1 iteração e 10 está na qualidade do prompt inicial: verbo de ação, stack + versões, arquivo alvo, tipo de retorno — e, quando ajudar, exemplos e restrições explícitas.

 objetivo

 contexto

 restrições

 iteração

Próxima aula → **TDD com Claude Code.**